

In the name of ALLAH

به امید نابودی رژیم نجس کودک کش



Amirkabir University of Technology  
(Tehran Polytechnic)

Deep Neural Network Course By Dr. Safabakhsh  
spring 2025

## Assignment (06.2-Bounes)

Mohammad Sadegh Eftekhari

403131014

[Sadegh.eft@aut.ac.ir](mailto:Sadegh.eft@aut.ac.ir)



## Contents

|                                                             |          |
|-------------------------------------------------------------|----------|
| <b>IMPORTS .....</b>                                        | <b>2</b> |
| <b>Q1: Data Preparation .....</b>                           | <b>3</b> |
| <b>Q2: Autoencoder Architecture.....</b>                    | <b>3</b> |
| Encoder .....                                               | 3        |
| Decoder.....                                                | 4        |
| <b>Q3: Autoencoder Training and Reconstruction.....</b>     | <b>4</b> |
| <b>Q4: The “GANomaly” Model Architecture .....</b>          | <b>5</b> |
| Shared Encoder Role .....                                   | 5        |
| Discriminator .....                                         | 5        |
| <b>Q5: Loss Functions.....</b>                              | <b>5</b> |
| Adversarial Loss ( $L_{adv}$ ) .....                        | 5        |
| Contextual Loss ( $L_{con}$ ) .....                         | 5        |
| Encoder Loss ( $L_{enc}$ ) .....                            | 6        |
| Final Objective Function.....                               | 6        |
| <b>Q6 &amp; Q7: Anomaly Score and Training Process.....</b> | <b>6</b> |
| Calculating the Anomaly Score .....                         | 6        |
| Training on Normal Data.....                                | 6        |
| <b>Q8: Evaluation and Results .....</b>                     | <b>7</b> |
| Visual Results: Successful Reconstructions .....            | 7        |
| Visual Results: Failed Reconstructions .....                | 7        |
| AUC-ROC Analysis.....                                       | 8        |

# IMPORTS

1. `torch/torch.nn/torch.optim`:  
The core PyTorch framework is used for building the Encoder, Decoder, and Discriminator models, defining the various loss functions (MSELoss, L1Loss, BCELoss), and managing the optimization process.
2. `torchvision`:  
Used to easily download and load the MNIST dataset and to apply necessary image transformations. Its utils also help in visualizing grids of images.
3. `matplotlib.pyplot/numpy`:  
This combination is our primary tool for visualization. We use it to display sample MNIST digits and to plot the final AUC-ROC curve.
4. `sklearn.metrics`:  
We use `roc_curve` and `auc` from Scikit-learn to calculate and evaluate the model's performance on the anomaly detection task.
5. `tqdm`:  
Provides a clean and convenient progress bar for monitoring the training loop for each epoch.

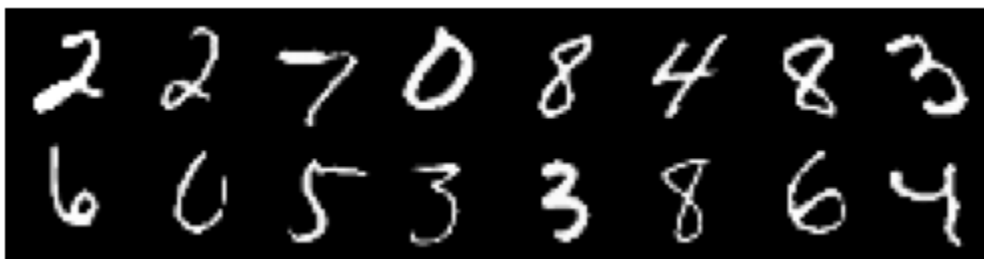
## Q1: Data Preparation

Our first step is to prepare the dataset. We use the torchvision library to download and load the standard MNIST dataset.

A transformation pipeline is defined to prepare the images for the model:

1. **transforms.ToTensor()**: This converts the PIL images into PyTorch tensors.
2. **transforms.Normalize((0.5,), (0.5,))**: This normalizes the tensor's pixel values from the [0, 1] range to the [-1, 1] range. This is a critical step to ensure the input data distribution matches the output of our generator's final nn.Tanh activation function.

Data loaders are created for both the training and test sets to efficiently batch and shuffle the data during model training and evaluation. A sample batch from the dataset is visualized below to confirm that the data has been loaded correctly.



## Q2: Autoencoder Architecture

As the foundation of the “GANomaly” model, we first design a convolutional autoencoder capable of reconstructing input images. The autoencoder consists of two symmetric components: an **Encoder** and a **Decoder**.

### Encoder

The Encoder's role is to “downsample” the input image into a compressed, low-dimensional latent representation. Our implementation uses a series of “nn.Conv2d” layers with “ReLU” activation functions to progressively extract features and reduce dimensionality.

**Input:** A 1\*28\*28 MNIST image.

**Layer 1:** A Conv2d layer reduces the image to 16\*14\*14.

**Layer 2:** A second Conv2d layer further reduces the feature maps to 32\*7\*7.

**Layer 3:** A final Conv2d layer with a 7\*7 kernel compresses the features into a 64\*1\*1 latent vector.

## Decoder

The Decoder is designed to be a mirror image of the Encoder. It takes the latent vector and upsamples it back to the original image dimensions using `nn.ConvTranspose2d` layers.

**Input:** The  $64 \times 1 \times 1$  latent vector from the Encoder.

**Layer 1:** A `ConvTranspose2d` layer with a  $7 \times 7$  kernel expands the vector to  $32 \times 7 \times 7$ .

**Layer 2:** A second `ConvTranspose2d` layer upsamples the features to  $16 \times 14 \times 14$ .

**Layer 3:** A final `ConvTranspose2d` layer reconstructs the  $1 \times 28 \times 28$  image.

**Output Activation:** A `nn.Tanh` function is used in the final layer to scale the output pixels to the  $[-1, 1]$  range, matching the normalized input images.

We then make a model, and put the 'Encoder' and the 'Decoder' in the same model to make an 'Auto-encoder'.

## Q3: Autoencoder Training and Reconstruction

To validate the autoencoder's architecture, we train it independently to ensure it can perform basic image reconstruction. The model is trained for 5 epochs using the Adam optimizer and the Mean Squared Error (`nn.MSELoss`) as the reconstruction loss function, which we refer to as  $L_{con}$ . This loss measures the pixel-wise difference between the original and reconstructed images.

After training, we pass a batch of test images through the autoencoder to visually inspect the reconstruction quality. As shown below, the model successfully learns to create clear and recognizable reconstructions of the input digits, confirming that the architecture is suitable for our task.

Original Images



Reconstructed Images



## Q4: The “GANomaly” Model Architecture

The complete “GANomaly” model builds upon the autoencoder by incorporating a generative adversarial framework. It consists of a **Generator** (our autoencoder) and a **Discriminator**. A key feature of this architecture is the shared use of the generator's encoder component.

### Shared Encoder Role

The encoder (`generator.encoder`) is used in three critical parts of the forward pass:

1. **Input Encoding:** It encodes the real input image  $x$  to produce its latent representation,  $z$ .
2. **Reconstruction Encoding:** It encodes the reconstructed image  $\hat{x}$  (generated by the decoder) to produce its own latent representation,  $\hat{z}$ .
3. **Discriminator Input:** The latent vectors  $z$  and  $\hat{z}$  are fed to the Discriminator to distinguish between representations of real and fake (reconstructed) data.

### Discriminator

The Discriminator is not a convolutional network that works on images. Instead, it is a simple Multi-Layer Perceptron (MLP) that operates on the *latent vectors* produced by the encoder. Its job is to determine if a given latent vector came from a real input image ( $z$ ) or a reconstructed one ( $\hat{z}$ ). It uses LeakyReLU activations and a final Sigmoid layer to output a probability score between 0 (fake) and 1 (real).

## Q5: Loss Functions

The “GANomaly” model is trained by optimizing a composite objective function made of three distinct loss components.

### Adversarial Loss ( $L_{adv}$ )

This loss drives the adversarial game. The generator tries to produce reconstructions ( $\hat{x}$ ) whose latent representations ( $\hat{z}$ ) are indistinguishable from the latent representations of real images ( $z$ ). The discriminator tries to tell them apart. We use `nn.BCELoss` for this task.

### Contextual Loss ( $L_{con}$ )

This is the reconstruction loss, which ensures the generator produces images that are visually similar to the input. We use `nn.L1Loss` to measure the difference between the input image  $x$  and the reconstructed image  $\hat{x}$ . An L1 loss is often preferred for encouraging sharper image generation compared to MSE.

## Encoder Loss ( $L_{enc}$ )

This loss measures the distance between the feature representations of the real and reconstructed images. By minimizing the “nn.MSELoss” between the latent vectors ‘z’ and “z\_hat”, we force the generator to learn the data distribution of the normal class in the latent space.

## Final Objective Function

The final generator loss is a weighted sum of these three components, allowing us to balance their respective contributions to the training process<sup>12</sup>:

$$LG = w_{adv}L_{adv} + w_{con}L_{con} + w_{enc}L_{enc}$$

The discriminator has its own adversarial loss, “ $mathcal{L}_D$ ”, which is the standard binary cross-entropy loss for classifying real and fake latent vectors.

## Q6 & Q7: Anomaly Score and Training Process

### Calculating the Anomaly Score

To determine if a new image is an anomaly, we need a quantitative score. We use a method based on the latent space representations, as suggested by the paper. The **anomaly score** is calculated as the L2 norm (Euclidean distance) of the difference between the latent vector of the input image (z) and the latent vector of its reconstruction (z\_hat).

$$Anomaly\ Score = ||z - z^{\wedge}||^2$$

The intuition is that if the input image is normal, the generator will reconstruct it well, leading to similar latent vectors and a low score. If the image is anomalous, the reconstruction will be poor, resulting in distant latent vectors and a high anomaly score.

### Training on Normal Data

The training process is the core of the anomaly detection setup. For each experiment, we designate one MNIST class (e.g., the digit '0') as **normal** and train the “GANomaly” model **exclusively on images from that class**. The training loop alternates between updating the generator and the discriminator:

- **Generator Step:** Minimize the combined loss “ $mathcal{L}_G$ ” to learn how to perfectly reconstruct normal images.
- **Discriminator Step:** Minimize “ $mathcal{L}_D$ ” to get better at distinguishing latent features of real normal data from reconstructed data.

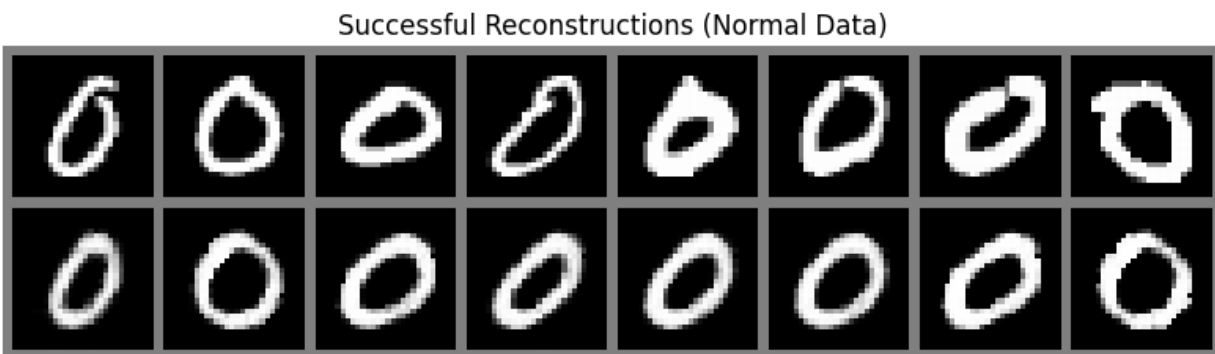


## Q8: Evaluation and Results

After training the model on a single normal class (here, '0'), we evaluate its performance on the entire MNIST test set, which contains both normal ('0') and anomalous ('1' through '9') samples.

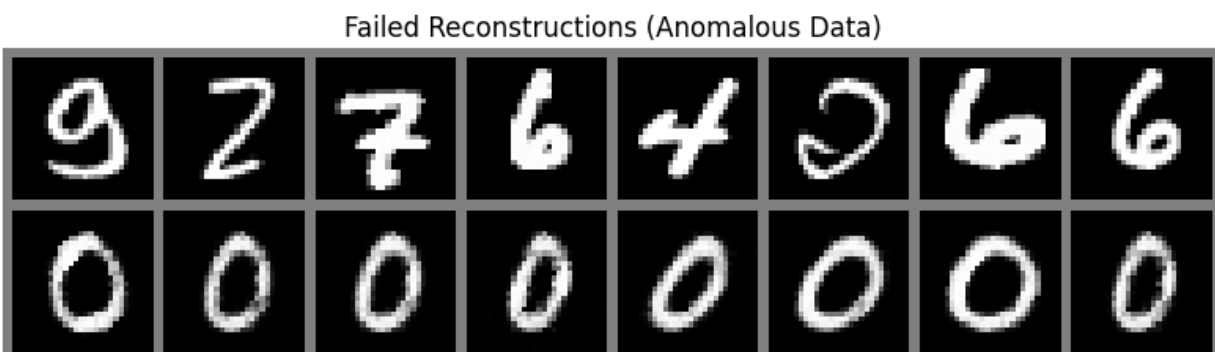
### Visual Results: Successful Reconstructions

When the model is given normal data (images of '0'), it reconstructs them accurately with minimal error. This demonstrates that the generator has successfully learned the data distribution of the normal class.



### Visual Results: Failed Reconstructions

Conversely, when the model is presented with anomalous data (any digit other than '0'), it fails to reconstruct them properly. The outputs are distorted, blurry, or resemble the normal class ('0'), leading to a high reconstruction error and a high anomaly score. This is the desired behavior and confirms the model's effectiveness.



## AUC-ROC Analysis

To quantitatively measure the model's ability to distinguish between normal and anomalous data, we calculate the **Area Under the Receiver Operating Characteristic Curve (AUC-ROC)**. The anomaly scores are used to rank the test samples, and the ROC curve plots the true positive rate against the false positive rate. An AUC of 1.0 represents a perfect classifier, while 0.5 represents a random one.

Our model achieves an **AUC of 0.72**, indicating a good level of accuracy in separating the normal class from all other anomalous classes.